Project Specification: Lights Out

GitHub repository: https://github.com/RickyRAV/cross-lights-out

2.1 Project Introduction

- **Team members**: Ricky Raveanu (RickyRAV), Nikola Petrovic (Nikola-Petro)
- **Working title**: Lights Out
- **Summary**: A multiplayer Formula 1 prediction game. Before each race weekend's session lockout, everyone in a friend group privately predicts the finishing order. Once the real session ends, the app auto-grades everyone against the actual result and updates a shared season leaderboard.
- **Target users / problem**: F1 fans who already trade predictions informally in group chats or casual bets, but have no real way to track who's actually right over a season. Lights Out turns that into an actual record: predictions are sealed so nobody can copy, results are graded automatically, and the leaderboard keeps score all season.

2.2 Chosen Framework

- **Framework**: React Native via Expo (managed workflow).
- **Rationale**: both of us already know React and TypeScript, so most of the learning curve is the mobile-specific stuff (native APIs, navigation, platform quirks), not the language itself.
- **Target platforms**: Android is developed and tested by both.

2.3 Planned Features (User Stories)

Each story links to the code that implements it. Mobile paths are relative to `apps/mobile/`, API paths to `apps/api/`.

1 **As a fan, I want to see the upcoming race with a countdown to lockout, so I know how long I have to predict.**

```
GIVEN the app is opened,
WHEN the Predict screen loads,
THEN the next race's name, date, and a live countdown are shown, sourced from
    our own API.
Done when the countdown reflects the real current-season schedule.
```

Implemented in: mobile `app/(tabs)/(predict)/index.tsx`, `components/prediction-summary.tsx`, `hooks/use-races.ts`, `hooks/use-clock.ts`, `lib/race-calendar.ts`; API `src/routes/races.routes.ts`, `src/services/race-calendar.service.ts`, `src/lib/jolpica-client.ts`.

2 **As a fan, I want to submit a sealed top-5 prediction before lockout, so I can compete without anyone copying my picks.**

```
GIVEN the prediction window is open,
WHEN I order 5 drivers and submit,
THEN it's saved to the shared backend, becomes uneditable after lockout,
AND stays hidden from friends until lockout passes (only a submitted/not-
    submitted flag is visible to them).
Done when a submitted prediction survives an app restart and is never returned
    to another user pre-lockout.
```

Implemented in: mobile `app/(tabs)/(predict)/prediction.tsx`, `components/prediction-editor.tsx`, `components/prediction-confirmation.tsx`, `hooks/use-predictions.ts`, `hooks/use-drivers.ts`, `store/prediction-draft.ts`; API

`src/routes/predictions.routes.ts`, `src/services/predictions.service.ts` (lockout and sealing rules, tested in `test/predictions.service.test.ts`).

3 **As a fan, I want to see which friends have already submitted, so I know who's still deciding.**

```
GIVEN the prediction window is open,
WHEN I view the race screen,
THEN each friend's submitted/not-yet status is shown without revealing picks.
Done when the status list updates as friends submit.
```

Implemented in: mobile `app/(tabs)/(predict)/race-detail.tsx`, `hooks/use-predictions.ts` (`useGroupPredictions`, refreshed every 15 s while open); API `src/services/predictions.service.ts` (picks are nulled before lockout).

4 **As a fan, I want a push notification before lockout, so I don't miss my chance to predict.**

```
GIVEN notification permission is granted and I haven't predicted yet,
WHEN lockout enters a configured reminder window,
THEN I get a local notification.
Done when it fires on Android, and permission-denied is handled without
    breaking the app.
```

Implemented in: mobile `lib/reminders.ts` (scheduling and cancelling), `hooks/use-reminders.ts` (permission state and the scheduler hook), `app/reminder-permission.tsx` (permission sheet), `app/_layout.tsx`, `app/(tabs)/settings.tsx`.

5 **As a fan, I want my prediction automatically scored against the real result, so I don't calculate points myself.**

```
GIVEN a race has finished,
WHEN the backend fetches the final classification from Jolpica-F1,
THEN every prediction for that race is scored and stored.
Done when the scoring function is unit-tested against known predicted/actual
    pairs.
```

Implemented in: API `src/lib/scoring.ts`, `src/services/grading.service.ts` (runs at startup and every 30 minutes), `src/lib/jolpica-client.ts`; tests `test/scoring.test.ts`, `test/grading.service.test.ts`.

6 **As a fan, I want a shared season leaderboard for my friend group, so I can see who's winning overall.**

```
GIVEN at least one race is scored,
WHEN I open the Leaderboard screen,
THEN members are ranked by total season points from the shared backend.
Done when a newly graded race is reflected without any manual step.
```

Implemented in: mobile `app/(tabs)/leaderboard.tsx`, `components/standing-row.tsx`, `lib/leaderboard.ts`, `hooks/use-leaderboard.ts`; component test `components/__tests__/standing-row.test.tsx`; API `src/routes/leaderboard.routes.ts`, `src/services/leaderboard.service.ts`.

7 **As a fan, I want to browse my data and submit my prediction even without a connection, so a bad signal doesn't cost me a race.**

```
GIVEN I've previously loaded the app online,
WHEN I open it offline,
THEN the calendar, my history, and the last-synced leaderboard are still
    viewable with a "last updated" indicator,
AND a prediction I submit offline is shown as "waiting to send" and is sent
    automatically once I'm back online.
Done when no screen shows a blank or broken state offline, and a queued
    prediction either reaches the backend before lockout or I'm clearly told it
    didn't count.
```

Implemented in: mobile `lib/query-client.ts` (persisted cache, network and auth wiring), `app/_layout.tsx` (`PersistQueryClientProvider`, resuming the queue),

`hooks/use-predictions.ts` (queued submission, lockout check, `useQueuedPrediction`), `hooks/use-online.ts`, `components/screen-status.tsx` ("last updated" banner), `components/prediction-summary.tsx` and `app/(tabs)/(predict)/index.tsx` ("waiting to send" and "didn't count"), `store/prediction-draft.ts`.

8 **As a fan, I want to create my friend group or join it with an invite code right after registering, so my predictions count toward a leaderboard with people I know.**

```
GIVEN I've just registered and I'm not in a group yet,
WHEN I create a group or enter a valid invite code,
THEN I'm in that group for good and appear on its leaderboard going forward.
Done when an invalid code is rejected without creating a membership, and a user
    who's already in a group can't create or join another one.
```

Implemented in: mobile `app/onboarding.tsx`, `hooks/use-groups.ts`, `store/onboarding.ts`, `app/_layout.tsx` (route guards), `app/(tabs)/settings.tsx` (invite code); API `src/routes/groups.routes.ts`, `src/services/groups.service.ts`, `src/lib/invite-code.ts`.

9 **As a fan, I want to look back at my past predictions next to the real results, so I can see where I gained or lost points.**

```
GIVEN I've predicted at least one race,
WHEN I open the History screen,
THEN every race I predicted is listed newest first with my top 5, the actual
    top 5, and the points I earned,
AND races that aren't graded yet are marked as pending instead of showing 0
    points.
Done when a race moves from pending to scored on its own once the backend
    grades it.
```

Implemented in: mobile `app/(tabs)/history.tsx`, `hooks/use-history.ts` (polls while a race is pending); API `src/routes/users.routes.ts`, `src/services/users.service.ts`.

10 **As a fan, I want to see what everyone in my group picked once predictions are closed, so we can compare who called the race best.**

```
GIVEN the lockout for a race has passed,
WHEN I open that race,
THEN I see the top 5 every group member picked,
AND after the race is scored, I also see the real result and each member's
    points.
Done when nobody's picks can be seen before lockout.
```

Implemented in: mobile `app/(tabs)/(predict)/race-detail.tsx`, `components/race-results.tsx`, `hooks/use-races.ts` (`useRaceResult`); API `src/services/predictions.service.ts`, `src/routes/races.routes.ts`.

2.4 Screen / Navigation Structure

- **Login / Register** (auth stack): email/password sign-in or account creation. Successful authentication opens the main tabs, or onboarding first if the user isn't in a group yet. An existing valid session skips Login. Signing out from Settings returns to Login.
- **Onboarding** (once, right after registering): create a friend group and share its invite code, or join one with a code. Every user is in exactly one group, and there's no leaving or switching, so a returning user goes straight to the tabs.
- **Predict** (tab, stack root): next race, countdown, prediction form or sealed "submitted" state → **Race Detail** (stack): full session info, friend status list, and (post-lockout) everyone's revealed picks/points.
- **Leaderboard** (tab): season standings.
- **History** (tab): past predictions, results, points earned.

- **Settings** (tab): notifications, your friend group's name and invite code, account.

Planned navigation flow:

```
App launch -> Session check
  No valid session -> Login <-> Register -> Group check (after authentication)
  Valid session ----------------------> Group check
Group check:
  No group yet -> Onboarding (create or join) -> Main tabs
  Already in a group ----------------------> Main tabs
Main tabs:
  Predict -> Race Detail (stack, back returns to Predict)
  Leaderboard
  History
  Settings -> Sign out -> Login
```

Key state: auth session, the user's friend group, cached calendar/results and queued offline submissions (TanStack Query), in-progress prediction draft (Zustand), notification permission status.

2.5 API and Data Sources

1. **Jolpica-F1** (`api.jolpi.ca/ergast/f1/`): a free REST API, no auth needed, compatible with the old Ergast API. Only our backend calls it, never the app directly. That way scoring can't be gamed client-side, and we're not stuck if Jolpica-F1 ever changes shape.
2. **Self-hosted backend** (running on Ricky's own server): the app's only backend. It holds predictions, friend groups, and the leaderboard, and proxies and caches the race calendar and final results from Jolpica-F1 so the app never talks to a third party directly. The app reads and writes to it through TanStack Query. After a race ends, the backend fetches the result from Jolpica-F1 itself and grades everyone.

Every screen talks only to our own backend. For auth we're using a simple self-issued JWT with email/password, nothing more since it's just our friend group and not a public app.

2.6 Native Feature Usage

We use **local notifications** through `expo-notifications` to remind a user to predict before lockout.

- **Where it's used**: whenever the app loads the race calendar, it schedules a reminder for the next race, set to fire a fixed time before that race's lockout. Submitting a prediction cancels the reminder for that race, so nobody gets nagged after they've already picked. Settings shows whether reminders are on. If the user hasn't decided yet, it asks for permission directly, and once permission is blocked it links to the system notification settings.
- **Why it matters**: lockout is the only deadline in the game, and missing it means zero points for that race. Race weekends move around the calendar and across time zones, so an in-app countdown alone isn't enough when the app is closed. A reminder on the lock screen is what actually keeps people from missing it.
- **Permissions**: the permission prompt is shown after onboarding, not on the very first screen, so the user already knows what the app is for when asked. On Android 13 and newer this is the runtime `POST_NOTIFICATIONS` prompt, and reminders go through their own Android notification channel. Expo Go, which we develop in, doesn't allow custom channels, so there the reminder falls back to the default channel. Granted, denied, and undetermined are all handled. If permission is denied, nothing is scheduled, Settings shows reminders as off, and the countdown on the Predict screen still works.
- **Why local instead of server push**: the lockout time is already on the device from the cached calendar, so the phone can schedule the reminder itself. That means no push server, no device tokens stored on our backend, and a reminder that still fires if the phone is offline when lockout gets close.

2.7 Offline / Persistence Strategy

- **Read, fully offline**: the calendar, history, and last-synced leaderboard are cached (TanStack Query's persisted cache), so they're browsable with no internet. We show a "last updated" timestamp so it's clear when it's stale.
- **Write, queued offline**: you can submit a prediction without a connection. The submission is queued on the device (TanStack Query's paused mutations, saved together with the persisted cache so the queue survives an app restart) and the Predict screen shows it as "waiting to send" instead of "submitted". As soon as the device is back online, the queue is sent automatically.

A prediction only counts once it reaches the backend before lockout. Before sending a queued prediction, the app checks the lockout time from the cached calendar. If lockout has already passed, it isn't sent, and the user is told the prediction didn't count because they were offline until after lockout. If the backend still rejects it, for example because the device clock was off, the user sees the same message instead of the error being swallowed. If a prediction is still waiting to send when the lockout reminder fires (2.6), the notification says so, so the user knows to get online.

The queue is sent while the app is open, or the next time it's opened. Sending while the app is fully closed would need background tasks, which Android doesn't run reliably, so we don't rely on them.

2.8 Technical Approach & Architecture

The mobile architecture follows the screens + hooks pattern: presentation (`app/` routes and `components/`), API access (`hooks/`, built on `lib/api.ts`), and local/UI state (`store/`). Screens compose these layers, keeping API access and state management outside the view code. Navigation uses an authentication stack and a one-time onboarding step, followed by bottom tabs, with a nested stack for Predict and Race Detail. The backend uses a three-layer architecture (routes/API, business logic, data access).

The app and the backend live in the same GitHub repo, set up as a pnpm workspace. The request and response types are defined once in `packages/shared` and used by both sides, so if the API changes shape, the app stops compiling instead of breaking at runtime.

```
apps/mobile/              Expo app (app/, components/, hooks/, lib/, store/)
apps/api/                 Fastify + Drizzle backend (routes/, services/, repositories/)
packages/shared/          request/response types used by both apps
docker-compose.yml        local dev (db + api)
docker-compose.prod.yml   deployed on Ricky's own server
```

Mobile's `store/` folder is Zustand, for local/UI state. The `hooks/` folder wraps TanStack Query calls to our own API. Screens are files in `app/` (Expo Router's file-based routing). Styling is done with NativeWind (Tailwind for React Native), with the colours defined once in `theme/colors.js`, which gives us full control over a custom dark racing theme instead of a default Material look. Shared building blocks (buttons, driver chips, status pills, form fields) are our own small components in `components/`.

The backend is built with Fastify and Drizzle on top of PostgreSQL, split into three layers: routes/API, business logic, and data access, so each part has one clear job. Incoming request bodies are validated with Zod before they reach the business logic layer, so bad input gets rejected early with a clear error instead of failing deeper in the code. It's deployed as a container on Ricky's own server, using the same docker-compose setup for local development and production, so both of us develop against one shared, always-on backend instead of two separate local databases. Screens don't call `fetch` directly, they always go through a query or mutation hook. For testing, the scoring algorithm and the backend services are covered by unit and route tests (Vitest, 88 tests), and the leaderboard row has a component test (jest-expo with React Native Testing Library). CI runs build, typecheck, lint, formatting and all tests on every pull request.

2.9 Changelog

- First draft of the specification, plus the backlog in `GHERKIN.md`. Set up the monorepo and deployed the API skeleton to Ricky's server, so the hosting setup was proven working before any features were built on top of it.

- The app no longer calls Jolpica-F1 directly (2.5). Only the backend talks to it, and it caches the race calendar and results for the app. Having the app call Jolpica-F1 for some data and our backend for the rest meant two different data sources in the client, and grading has to happen on the server anyway so it can't be gamed.
- Finished the backend: auth, friend groups, sealed predictions, the race calendar cache, automatic grading, and the leaderboard. Checking every planned screen against the API showed a few endpoints the screens would need but that weren't planned, so we added them: listing your groups, fetching your own prediction, your identity and prediction history, a single race's result, and the number of races scored on the leaderboard.
- Made the auth and onboarding navigation clearer (2.4) and described the mobile architecture in more detail (2.8).
- Each user is now in exactly one friend group, created or joined during onboarding right after registering (story 8, 2.4). Supporting several groups per user would have meant a group picker on almost every screen, while the whole point of the app is one leaderboard with your friends.
- Added user stories 9 (history) and 10 (seeing everyone's picks after lockout), expanded 2.6 with where notifications are used and how permissions are handled, and changed offline submissions to be queued and sent automatically once the device is back online (story 7, 2.7). The first plan only kept a local draft that had to be submitted again by hand, which is easy to forget right before lockout.
- Built the app with Expo Router, so screens live in `app/` instead of a `screens/` folder (2.8). Dropped gluestack-ui: NativeWind plus a handful of our own components covered every screen, so the extra dependency wasn't needed (2.8).
- Added a season driver list to the backend (`GET /drivers`, including each driver's current team) so the prediction form only offers real driver codes and can colour them by team.
- Built the offline mode as planned (2.7). The cache and the submission queue are stored on the device with expo-sqlite, and unsent prediction drafts are kept too. The lockout reminder now warns when a prediction is still waiting to send (2.6, 2.7).
- Settings asks for notification permission directly when the user hasn't decided yet, instead of always sending them to the system settings (2.6).
- Race Detail shows the official race result next to everyone's picks once the race is graded (story 10).
- Added automated tests on the app side (a leaderboard component test) and linked every user story to the code that implements it (2.3).